

Zr3d Code Architecture

Nasr M. Ghoniem

August 8, 2026

Abstract

This document describes the architecture of the `Zr3d_ghoniem` code: the software that solves the spatially-resolved cluster-dynamics (SR-CD) model of irradiated zirconium, and the zero-dimensional `ZrMicro` model against which it is verified. The code is not a single program. It is a two-repository system driven from a Jupyter notebook, in which a Python orchestration layer, a C++ stiff-ODE integrator, and a C++ finite-element field solver each own one part of the problem and exchange state through well-defined interfaces. Section 1 describes the workflow, Section 2 enumerates the components and their locations, Section 3 lists the inputs with typical values and units, Section 4 gives the call graph, and Section 5 documents the classes and data structures with references to the governing equations of the Phase-4 D1/M1 deliverable (`Docs/Formulation/GW_Phase4_D1M1_NG.pdf`, cited throughout as Eq. (n)). Sections 1–5 describe `Zr3d_ghoniem` as it now stands; Section 6 then separates what was inherited from the original `MoDELib3` from what was added, at the level of both the equations and the input data. Appendix A is the operating manual.

Contents

1	General Architecture	1
1.1	The problem the architecture has to solve	1
1.2	Two execution routes	2
1.3	The notebook workflow, cell by cell	2
1.4	The dose march	3
2	Code Components	4
2.1	Repositories	4
2.2	Python components (<code>Fluor_Zr/ZrMicro/py_utils/</code>)	4
2.3	C++ components	5
2.4	How the components interact	6
3	Code Input	7
3.1	ZrMicro workbook: <code>ZrMicro/input/Zr_input_parameters.xlsx</code>	7
3.2	Notebook run controls	7
3.3	MoDELib3 material file: <code>Library/Materials/Zr3d_ghoniem.txt</code>	9
3.4	MoDELib3 simulation files	10
3.5	Unit systems, and the conversions between them	10
4	Code Flowchart	11
4.1	The inner loop of Route B	12

5	Code Classes and Data Structures	12
5.1	The two state vectors	12
5.2	Python classes	13
5.3	C++ structures: ZrMicro	14
5.4	C++ classes: MoDELib3	14
5.5	Relationships among the structures	16
6	Changes and Differences from Upstream	16
6.1	Which upstream — a correction	16
6.2	What MoDELib2-NNL contains	17
6.3	Equations added	18
6.4	Corrections to pre-existing code	18
6.5	Input data: the mobile block	19
6.6	Input data: the immobile block	20
6.7	What is unchanged	20
A	Code Manual	21
A.1	Prerequisites	21
A.2	Building	21
A.3	Running	22
A.4	Post-processing	22
A.5	Changing things	22
A.6	Troubleshooting	23
A.7	Reproducibility	24

1 General Architecture

1.1 The problem the architecture has to solve

The physics splits cleanly into two time scales, and the architecture is a direct expression of that split.

The *mobile* species — vacancies, single interstitials, di- and tri-interstitials — relax to their local sink field in of order milliseconds. The *immobile* species — the four loop families — evolve over dose steps of order 10^7 s. The ratio is $\sim 10^{10}$. Integrating both with one time integrator would be wasteful at best and intractable at worst.

The deliverable therefore poses the mobile species as a *quasi-steady* reaction–diffusion boundary-value problem with no time-derivative term at all, Eq. (1),

$$0 = -\nabla \cdot \mathbf{J} + P_M + \frac{1}{2} \mathbf{c}_M^T R_2 \mathbf{c}_M + (E - D) \mathbf{c}_M + S_{vL}, \quad (1)$$

and marches the immobile species pointwise in dose against the frozen mobile field, Eq. (34) for the density and Eq. (39) for the content. This is an operator-split quasi-steady-state approximation (QSSA): within one dose step the fast manifold is solved to steady state, then the slow manifold is advanced with the fast one held fixed.

1.2 Two execution routes

That split can be realized in two ways, and *both exist in this codebase*. They share the material definitions and the physics, but not the driver. Confusing them is the single most common source of misinterpretation, so they are separated explicitly here.

Route A — notebook-driven coupled march. Driven by `ZrMicro/code/coupled_0d_3d_ZrMicro.ipynb`. Python owns the dose loop. Within each step it calls `MoDELib3` to solve the fast mobile BVP on the finite-element mesh, then advances the slow immobile state *using the ZrMicro C++ integrator* with the mobile species pinned (`freeze_mobile=1`), one OpenMP batch case per quadrature point. The immobile physics executed here is `ZrMicro`’s, evaluated at many spatial points.

Route B — monolithic MoDELib3 run. Driven by `tutorials/zrmicro_coupled/clean_run.sh`, which invokes `microstructureGenerator` and then `DDomp`. Everything happens inside `MoDELib3`: `ClusterDynamicsFEM::solveMobileClusters()` performs the Newton solve of Eq. (1), and `solveImmobileClusters()` marches the loop families node by node. No Python is involved at run time. This is the route that produced the 30 dpa verification results, and it is the route the `Zr3d_ghoniem` branch modifies.

Route A is the *coupling harness*: it tests the splitting and lets the 0-D physics be exercised in space. Route B is the *3-D code proper*: it is where the SR-CD equations of deliverable Sec. 2.2 are actually implemented. The remainder of this document covers both, labelling each component by the route in which it participates.

1.3 The notebook workflow, cell by cell

The notebook is the orchestrator for Route A and the entry point a user actually touches. Its structure is linear and each stage has one responsibility.

1. **Formulation (markdown, cells 0–5).** The 3-D SR-CD model, the reduced 0-D model, the correspondence between them, and the numerical methods. No code; this is the specification the rest of the notebook implements.
2. **Bootstrap (cell 7).** Resolves `ZrMicro/` from the notebook location — tolerating launch from `code/`, from `ZrMicro/`, or from the repository root — puts it on `sys.path`, and imports the entire Python layer: `InputData`, `ReactionRates`, `RateEquations`, `cpp_bridge`, `post_process`, `visualization`, `modelib_coupling`, `modelib_export`, `modelib_fem`.
3. **Build management (cells 8–10).** Two independent builds are ensured before anything runs. `ensure_zrmicro_solver()` locates `solver.exe`, configuring and building through CMake if absent. `ensure_modelib()` constructs a `MoDELibFEMSolver` and, if `DDomp` is missing, runs the WSL build script. On Windows `MoDELib3` is built inside WSL and its `DDomp` is a Linux ELF, so it is invoked through `ws1.exe` with Windows paths translated to `/mnt/<drive>/...`. *Both build paths verify artifacts on disk rather than trusting exit status*, because the build script terminates in a `|| echo WARN` and returns zero even when it produced nothing.
4. **User controls (cell 12).** One block holds every knob: the dose schedule (`DOSE_START`, `DOSE_INTERVAL`, `DOSE_MAX`), the physical conditions (T , G , σ_n) applied to *both* codes, the spatial discretization, the runtime flags, the 0-D solver configuration, and the fitted `PARAMETER_OVERRIDES` set. The overrides are pinned at full precision rather than inherited from the workbook, because the workbook has drifted since the reference run: 28 parameters differ and 11 are now absent from it entirely.
5. **Model construction (cell 13).** `InputData` reads the workbook; each override is routed automatically to whichever sheet already defines the key; `calculate_derived_parameters()` recomputes the derived quantities; then `ReactionRates` and `RateEquations` are built on top. Finally `collect_solver_args()` flattens the whole parameter set into the `--key=value` command line the C++ integrator consumes.

6. **Consistency check (cell 14).** Parses MoDELib3’s material file and compares it against the ZrMicro conditions field by field — dose rate, species vector, and the four DAD capture efficiencies of Eq. (15) against the 0-D symmetric forms Eqs. (63)–(64). It also reports where the two models are known *not* to correspond, so that a residual is not mistaken for a bug.
7. **Run directory (cell 16).** A timestamped, git-hash-tagged output directory, so every figure and every number is traceable to a commit.
8. **0-D reference run (cells 18–20).** The full ZrMicro integration, used both as the physical reference and as the source of the initial immobile state for the march.
9. **Spatial grid and backend selection (cells 22–23).** The grid is built in units of the depletion length $\ell = 1/\sqrt{Z_N \rho_N}$, the domain spanning DOMAIN_IN_ELL multiples of it. The fast-solve backend is then chosen: modelib if DDomp is built *and* the simulation directory is seeded, otherwise an analytic placeholder $C_M^*(x) = [1 - e^{-x/\ell}] C_M^{\text{bulk}}$. The placeholder is loudly labelled as not being a FEM result — it scales all four species by the same factor, which cannot represent species-dependent depletion lengths that here differ by four orders of magnitude.
10. **Operator-split march (cell 25).** The core loop, discussed below.
11. **Post-processing (cells 26–33).** Regression against the 0-D reference, 3-D field plots, grain-boundary profiles, and a provenance manifest.

1.4 The dose march

Stripped to its essentials, the march is:

```
Y = [pack_y0(CM0[q], immobile_0, rho_seed) for q in range(N_POINTS)]

for n in range(len(DOSE_SNAPSHOTS) - 1):
    Qv = q_per_variant_from_state(Y) # loops -> (N, r) per family
    CM = fast_solve_field(dose[n], Qv) # FAST: MoDELib FEM mobile BVP
    for q in range(N_POINTS):
        Y[q][0:4] = CM[q] # inject frozen mobile field
    Y = march(Y, times[n], times[n+1]) # SLOW: ZrMicro batch, freeze_mobile=1
    history[dose[n+1]] = Y
```

Three properties of this loop are worth stating explicitly.

The coupling is two-way but staggered. The loop state sets the sink field that the mobile solve sees (through `q_per_variant_from_state`, which reduces the loop populations to a number density and a mean radius per family); the resulting mobile field then drives loop growth over the step. Neither is solved simultaneously with the other.

The immobile march is itself substepped. `march()` divides the dose step into intervals no longer than `MAX.SUBSTEP_S` and calls `run_immobile_step` on each. A point that fails to integrate retains its previous state and is reported rather than aborting the run.

Every quadrature point is one batch case. `run_immobile_step` builds one `--key=value` case per point, all sharing the material parameters in `base_cli` (valid because T , σ and the bias factors are spatially uniform) and differing only in the initial state and the integration window. They are integrated concurrently by the OpenMP batch mode of the C++ solver in a single subprocess. This is what makes the pointwise march affordable: it amortizes the process-spawn cost, which dominates on Windows.

2 Code Components

2.1 Repositories

The code spans the two repositories of Table 1.

Table 1: The two repositories.

repository	remote	role
Fluor_Zr	Ghoniem/Fluor_Zr	0-D model, orchestration, post-processing, doc
ileMoDELib3	ileGhoniem/MoDELib3, branch ilezr3d_ghoniem	3-D FEM SR-CD solver

The MoDELib3 work lives on the branch `zr3d_ghoniem` of a fork. The cluster-dynamics upstream is `m1m335/MoDELib-fullCD`, *not* `m1m335/MoDELib2-NNL` — see Section 6. Po’s original material definition `Zr4.txt` is left untouched — the reconciled definition is the separate file `Zr3d_ghoniem.txt`.

2.2 Python components (`Fluor_Zr/ZrMicro/py_utils/`)

Table 2 lists the Python layer. It carries no physics that the C++ layer does not also carry: its role is to read the inputs, derive the constants, orchestrate, and post-process.

Table 2: Python layer. “Route” marks which execution route uses the module: A = notebook coupled march, B = monolithic MoDELib3 run, P = post-processing common to both.

module	route	role
<code>input_data.py</code>	A	InputData : reads the three-sheet workbook, supplies defaults for missing entries, and computes every derived quantity — jump frequencies, equilibrium concentrations, length scales, generation-rate partition, DAD bias factors.
<code>reaction_rates.py</code>	A	ReactionRates : all pairwise reaction rates $R_{a,b}$, sink concentrations Eq. (66), loop absorption Eqs. (83)–(89), thermal emission, nucleation, annealing, and coalescence. State is cached per ODE call.
<code>rate_equations.py</code>	A	RateEquations : the 19-equation right-hand side, initial conditions, loop radii, strain rates, hardening. Mirrors the C++ RHS exactly.
<code>cpp_bridge.py</code>	A	Flattens the parameter set into <code>--key=value</code> arguments, launches <code>solver.exe</code> (single or <code>--batch_file</code> mode), parses stdout into a solution matrix.
<code>simulation.py</code>	A	ZrMicroSimulation : the SciPy LSODA fallback path and an interrupt guard. Retained as a reference implementation.
<code>modelib_coupling.py</code>	A	The operator-split driver. <code>pack_y0</code> assembles the length-19 state per point; <code>build_immobile_cases</code> sets <code>freeze_mobile=1</code> and the step window; <code>run_immobile_step</code> runs the batch and unpacks it.

module	route	role
<code>modelib_fem.py</code>	A	<code>MoDELibFEMSolver</code> : writes the loop state into <code>MoDELib3</code> 's microstructure inputs, invokes <code>DDomp</code> (through <code>ws1.exe</code> when required), and reads the mobile field back at the quadrature points.
<code>modelib_export.py</code>	A	Maps the lumped 0-D loop families onto the three $\langle a \rangle$ prismatic variants using the resolved-stress weights $w_k(\sigma)$ of Eq. (36).
<code>modelib_fields.py</code>	B, P	Reads <code>evl/*.txt</code> and <code>evl/cdNodes.txt</code> ; plane-cut field panels and loop-platelet overlays.
<code>modelib_gb.py</code>	B, P	Grain-boundary profiles of density, content, size and mobile species. Loop diameters are formed from binned mean content and density, not by averaging nodal diameters.
<code>post_process.py</code>	A, P	Derived macroscopic quantities: loop radii and densities in physical units, growth and creep strains, hardening, conservation diagnostics.
<code>visualization.py</code>	A, P	<code>ZrMicroVisualizer</code> : the figure suite, the run directory, and <code>provenance.md</code> .
<code>pre_process.py</code>	A	Input-file resolution, setup validation, parameter-range warnings.
<code>fit_loops.py</code> , <code>fit_vacancy_loops.py</code> , <code>fit_strain_factors.py</code>	—	Offline parameter identification against the experimental sheets.
<code>transport_audit.py</code>	—	Diagnostic harness.

2.3 C++ components

Table 3 lists the compiled layer. The two C++ codes are entirely independent builds — different toolchains, different libraries, no shared headers — and communicate only through the interfaces of Sec. 2.4. Entries marked **New in Zr3d_ghoniem** have no counterpart in the original `MoDELib3`; Sec. 6 separates the inherited code from the added code in full.

Table 3: C++ layer across both repositories.

file	route	role
<code>Fluor_Zr/ZrMicro/ code/solver.cpp</code>	A	Entry point. <code>integrate_one()</code> performs one CVODE integration; <code>run_batch()</code> reads a batch file and integrates all cases concurrently under OpenMP, each with its own <code>SUNContext</code> (SUNDIALS objects are never shared across threads).
<code>cpp_utils/parameters.h</code>	A	The <code>Parameters</code> struct: every rate constant, bias factor, generation rate and length scale, pre-computed so the RHS performs arithmetic only.

file	route	role
cpp_utils/rate_equations.cpp	A	The 19-equation RHS, including the <code>freeze_mobile</code> branch that zeroes the mobile derivatives <i>after</i> every reaction term has used them.
cpp_utils/CMakeLists.txt	A	C++17, SUNDIALS 7.1.1, OpenMP auto-detected.
MoDELib3/ (branch zr3d_ghoniem)		
tools/DDomp/DDomp.cpp	B	The runner. Constructs <code>DislocationDynamicsBase</code> , reads <code>evl/</code> , constructs <code>DefectiveCrystal</code> , calls <code>runSteps()</code> .
tools/MicrostructureGenerator/	B	Writes the initial configuration <code>evl_0.txt</code> . With <code>useDislocations=0</code> this is an empty 20-byte file, which DDomp nevertheless requires to exist.
include/ClusterDynamics/ClusterDynamics.h, src/.../ClusterDynamics.cpp	B	ClusterDynamics : the <code>MicrostructureBase</code> subclass. Owns the parameters and the FEM object, and implements the microstructure interface — <code>solve()</code> , <code>getDt()</code> , <code>output()</code> , stress and displacement contributions.
include/ClusterDynamics/ClusterDynamicsFEM.h, src/.../ClusterDynamicsFEM.cpp	B	ClusterDynamicsFEM : the numerical core. Assembles the weak forms, runs the mobile Newton iteration, and marches the immobile families.
include/ClusterDynamics/ImmobileSinks.h	B	New in Zr3d_ghoniem . An <code>EvalFunction</code> supplying the loop absorption sink acting on the mobile species, Eqs. (11)–(14).
include/ClusterDynamics/SecondOrderReaction.h	B	EvalFunction evaluating the binary reaction block R_2 from the local mobile field.
include/ClusterDynamics/FixedDirichletSolver.h	B	Linear solver wrapper that eliminates the constrained degrees of freedom, templated on a direct and an iterative backend.
include/DislocationDynamicsBase/ClusterDynamicsParameters.h, src/.../ClusterDynamicsParameters.cpp	B	ClusterDynamicsParameters : everything read from the material file and every matrix derived from it.

2.4 How the components interact

Three interfaces carry all cross-component state.

Interface 1: Python → ZrMicro C++ (Route A). A flat `--key=value` command line, one token per scalar parameter, produced by `collect_solver_args()`. Results return as a whitespace-delimited matrix on stdout. In batch mode the file holds one case per line and each block of output is prefixed `=== CASE <i> status=<s> ===`. The interface is deliberately textual: it has no build-time coupling, so either side can be rebuilt independently.

Interface 2: Python ↔ MoDELib3 (Route A). The file system. `MoDELibFEMSolver.write_sink_field()` writes the loop state into `inputFiles/aLoopsDensity.txt` and `inputFiles/frankLoopsDensity.txt` using `modlibUtils`; DDomp is then run on the directory; the mobile field is read back from the output. When MoDELib3 is built in WSL and Python runs on Windows, this crosses an OS boundary, which is why path translation lives in `windows_to_wsl_path()`.

Interface 3: MoDELib3 → post-processing (Route B). `ev1/ev1.N.txt`, holding 12 columns per finite-element node (four mobile species, four loop densities, four loop contents), plus `ev1/cdNodes.txt`, holding the node coordinates. The second file is essential and is written by `ClusterDynamicsFEM::writeNodePosition` the cluster-dynamics trial functions live on second-order elements — 24 115 nodes for a 3392-node, 15 702-tetrahedron mesh — so the field rows do *not* correspond to the mesh vertices and the ordering cannot be reconstructed externally.

3 Code Input

Input is spread over four files in two formats. The ZrMicro side is an Excel workbook; the MoDELib3 side is three plain-text `key=value`; files.

3.1 ZrMicro workbook: `ZrMicro/input/Zr_input_parameters.xlsx`

Three parameter sheets are read — Tables 4, 5 and 6. The remaining fifteen sheets hold digitized experimental data (Northwood, Gilbert, Jostsons, Cockeram, Tournadre, Harte and others) used only by the offline fitting utilities, never by the solver.

Table 4: Sheet **Material Environment**. Irradiation conditions and the sink microstructure.

symbol	quantity	units	typical value
ρ	network dislocation density	m^{-2}	1.5×10^{14}
ρ_{300}	network density anchor at 300 K	m^{-2}	1.0×10^{14}
ρ_{600}	network density anchor at 600 K	m^{-2}	2.0×10^{14}
d	grain diameter	m	1.0×10^{-5}
r_p	precipitate radius	m	5.0×10^{-8}
N_P	precipitate number density	m^{-3}	1.0×10^{20}
T	irradiation temperature	K	573
G	dose rate	dpa s^{-1}	1.0×10^{-7}
σ_n	applied normal stress	Pa	0 (verification runs)
σ_h	hydrostatic stress	Pa	0

3.2 Notebook run controls

Table 7 lists the remaining inputs. These are not material parameters; they configure the run.

3.3 MoDELib3 material file: `Library/Materials/Zr3d_ghoniem.txt`

Everything outside the `# ClusterDynamics` section — elasticity, mobility laws, gamma-surface — is inherited from `Zr4.txt` unchanged. The cluster-dynamics keys are listed in Table 8; the simulation-level files in Table 9. Roughly a third of the cluster-dynamics keys are reset from Po’s values and the rest are new; Sec. 6 gives the before-and-after.

Table 5: Sheet `PhysicalProperties`. Entries left at zero in the workbook are filled from the defaults in `InputData._set_default_values()`; the value shown is the one actually used at $T = 573$ K.

symbol	quantity	units	value
a	lattice parameter	m	3.23×10^{-10}
b_a	Burgers vector, $\langle a \rangle$	m	3.23×10^{-10}
b_c	Burgers vector, $\langle c \rangle$	m	5.15×10^{-10}
Ω	atomic volume	m ³	1.2×10^{-29}
z_c	coordination number	—	48
<code>recom</code>	recombination volume factor	—	0.5
E_i^m	interstitial migration energy	eV	0.759101 (fitted)
E_{2i}^m	di/tri-interstitial migration energy	eV	1.36292 (fitted)
E_v^m	vacancy migration energy	eV	1.2
E_v^F	vacancy formation energy	eV	1.6
E_i^F	interstitial formation energy	eV	3.0
E_{2i}^b	di-interstitial binding energy	eV	0.957033 (fitted)
E_{3i}^b	tri-interstitial binding energy	eV	2.0
ν_i, ν_v	attempt frequencies	s ⁻¹	5×10^{12}
μ	shear modulus	Pa	3.0×10^{10}
ν	Poisson ratio	—	0.34
ε_{vL}	cascade efficiency, vacancy loops	—	0.005 (fitted)
ε_{iL}	cascade efficiency, $\langle a \rangle$ loops	—	0.00211286 (fitted)
ε_{2i}	cascade efficiency, di-interstitials	—	0.01 (fitted)
ε_{3i}	cascade efficiency, tri-interstitials	—	0.01 (fitted)

Table 6: Sheet `ModelParameters`, and the fitted overrides applied on top of it by the notebook. The overrides reproduce the reference run `output/20260622.144021.7959445`.

symbol	quantity	units	value
Z_N	network bias for interstitials	—	1.01
Z_{iL}	interstitial-loop bias	—	1.34408
Z_{vL}	vacancy-loop bias	—	1.02
Q	absorption efficiency, $\langle c \rangle$ loops	—	0.287931
A_a, A_c	strain factors, $\langle a \rangle$ and $\langle c \rangle$	—	1.0
δ_{DAD}	vacancy DAD anisotropy	—	0.107886
$\delta_{\text{DAD}}^i(300 \text{ K})$	interstitial DAD at 300 K	—	0.153343
$\delta_{\text{DAD}}^i(600 \text{ K})$	interstitial DAD at 600 K	—	0.0640797
c_{LL}^a, c_{LN}^a	coalescence coefficients, $\langle a \rangle$	—	120.577, 1130.85
c_{LL}^c, c_{LN}^c	coalescence coefficients, $\langle c \rangle$	—	0.162483, 2.95829
κ_{LL}, κ_{LN}	Avrami overlap factors	—	2.0, 0.4
n_{iL}^{nuc}	interstitials per cascade $\langle a \rangle$ loop	—	300
n_{vL}^{nuc}	vacancies per cascade $\langle c \rangle$ loop	—	400
τ_{vL}^0	vacancy-loop dissolution prefactor	s	1.05145
E_{vL}^a	vacancy-loop dissolution energy	eV	0.470079
$c_{\rho N}, k_{\rho N}^{\text{rec}}$	network source/recovery	—	0.001, 3.47726×10^{-5}
$\alpha_\bullet$	obstacle strengths (7 families)	—	0.1–0.3

Table 7: Run controls, cell 12 of the coupling notebook.

control	units	typical value
DOSE_START / DOSE_INTERVAL / DOSE_MAX	dpa	0.01 / 0.10 / 1.00
N_POINTS	—	25 quadrature points
DOMAIN_IN_ELL	ℓ	12.0
MAX_SUBSTEP_S	s	5×10^4
FAST_SOLVE_BACKEND	—	auto modelib placeholder
rtol / atol	—	10^{-6} / 10^{-20}
solver_method	—	CVODE, BDF, dense

Table 8: Cluster-dynamics keys. Values are as reconciled with the 0-D model at $T = 573$ K.

key	units	value
doseRate_dpaPerSec	dpa s ⁻¹	1.0e-7
atomicVolume_SI	m ³	1.2e-29 (<i>optional key; without it the code falls back to the lattice cell volume, which for HEX holds two atoms</i>)
b_SI	m	0.3233e-9
mobileSpeciesVector	—	-1 1 2 3
mobileSpeciesCascadeFractions	—	0.995 0.97788714 0.005 0.00333333
mobileSpeciesEnergyFormation_eV	eV	1.6 3.0 6.0 8.0
mobileSpeciesEnergyMigration_eV	eV	4 × 6 tensor components; 1.2 for v, 0.759101 for i/2i/3i, isotropic
mobileSpeciesD0_SI	m ² s ⁻¹	5.21645e-7, isotropic, all species
reactionPrefactorMap	—	10 pairs; see Sec. 5
Eb_eV	eV	5.0 5.0 0.957033 2.0
otherSinks_SI	m ⁻²	1.88e14 1.8988e14 1.8988e14 1.8988e14
immobileSpeciesVector	—	-1 1 1 1 (polarity)
immobileSpeciesBurgers	—	lattice components, 3 × 4
loopCascadeFractions	—	0.005 0.000704 0.000704 0.000704
nNuc	—	400 300 300 300
cLL / cLN	—	0.162483 120.577×3 / 2.95829 1130.85×3
kappaLL / kappaLN	—	2.0 / 0.4
tau0_vLoop_SI / Ea_vLoop_eV	s / eV	1.05145 / 0.470079
rhoNetwork_SI	m ⁻²	1.88e14
dadAnisotropy (p_m)	—	1.178808 0.913720×3
dadZ0 (Z_m^0)	—	0.939836 1.015504×3
loopSinkScale	—	0.291528 0.792317×3
concentrationFloor	—	1.0e-20
nmin / nmax	—	100 40 40 40 / 800 300 300 300
r_min	m	1.0e-9 (all families)

3.4 MoDELib3 simulation files

3.5 Unit systems, and the conversions between them

The two codes do not share a unit system (Table 10), and most of the reconciliation defects found so far were unit errors. The conversions are therefore stated explicitly.

With $\Omega_{b^3} = \Omega/b^3 = 0.355105$, the density conversion is $N_{0D} = N_{3D} \Omega_{b^3}$, and the defects per loop follow as $m_k = c_k/(N_k \Omega)$. One dose step of 1 dpa at $G = 10^{-7}$ dpa s⁻¹ is 6.95869×10^{19} MoDELib3 time units.

Table 9: tutorials/zrmicro-coupled/inputFiles/. Only the entries that matter for a cluster-dynamics-only run are shown.

file / key	units	value
polycrystal.txt		
materialFile	—	Zr3d_ghoniem.txt
absoluteTemperature	K	573
meshFile	—	unitCube_15K.msh
F (deformation gradient)	b	≈ 3093 per side $\approx 1 \mu\text{m}$
periodicFaceIDs	—	empty: every face is grain boundary
DD.txt		
useClusterDynamics / useClusterDynamicsFEM	—	1 / 1
useDislocations	—	0 (loops are continuum fields)
Nsteps	—	31
startAtTimeStep	—	0 (<i>never -1: that silently restarts from a stale evl_N and a</i>
dtMax	b/c_s	$6.95868964\text{e}+19 \equiv 1 \text{ dpa at } G = 10^{-7}$
outputFrequency	—	1

Table 10: Unit conventions.

quantity	ZrMicro	MoDELib3
length	m	$b = 3.233 \times 10^{-10} \text{ m}$
time	s	$b/c_s = 1.437052 \times 10^{-13} \text{ s}$
mobile concentration	atom fraction	atom fraction (same)
loop number density	per atom	per b^3
loop content	per atom	volume fraction (numerically the same)

grey for products. Solid arrows are calls; dashed arrows are data written to or read from disk.

Component call graph. Solid arrows are calls, dashed arrows are file input/output. Route A drives the fast mobile solve through **MoDELib3** and the slow immobile march through the **ZrMicro** integrator; Route B runs both inside **MoDELib3**. The two routes share the material definitions and the post-processing layer.

4.1 The inner loop of Route B

Figure 1 expands the single MoDELib3 dose step, which is where the SR-CD equations are actually solved.

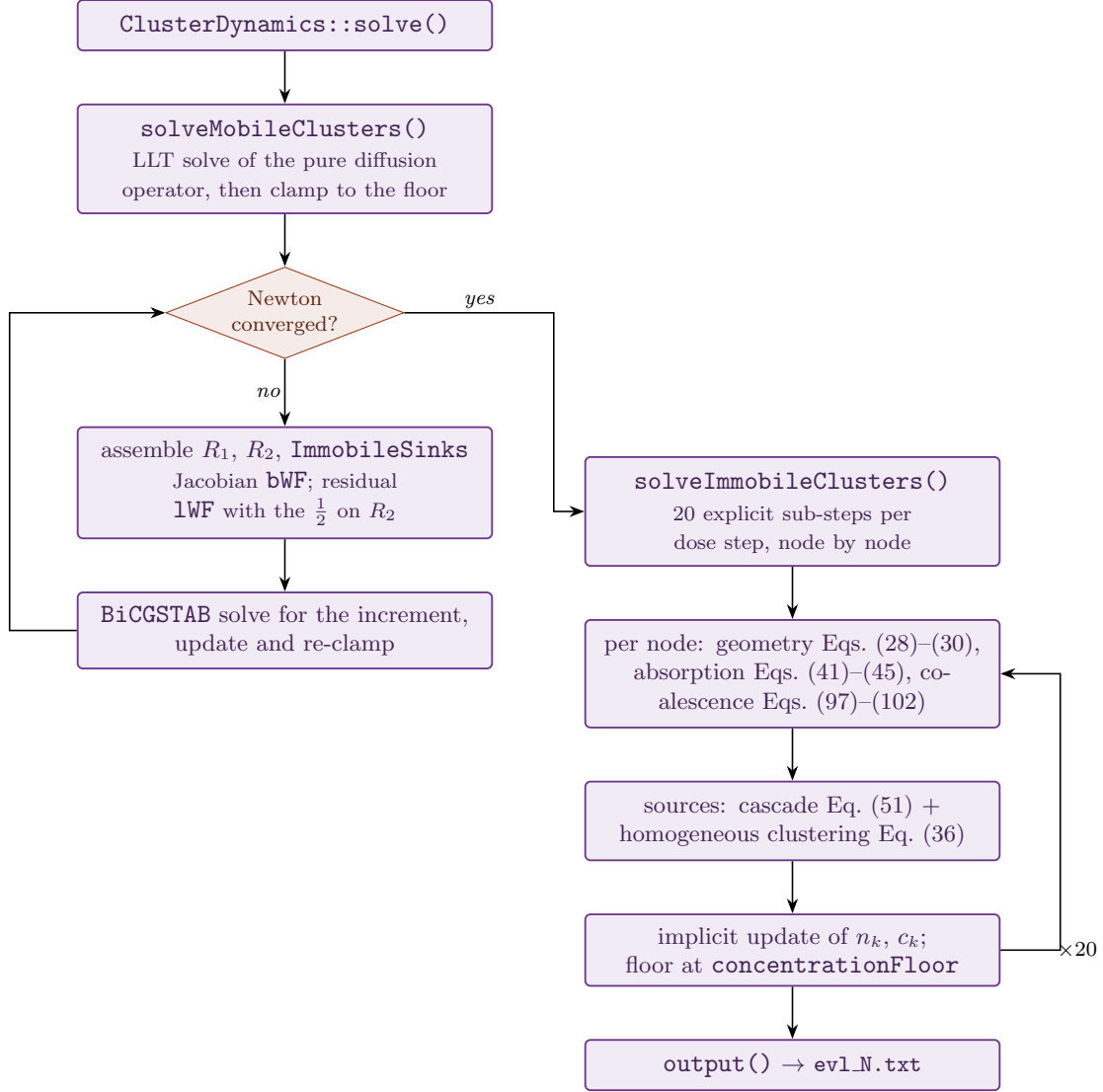


Figure 1: One dose step inside MoDELib3. The mobile Newton iteration runs to a relative tolerance of 10^{-5} ; the immobile march then advances the loop families against the converged, frozen mobile field. The linear losses (coalescence, dissolution, emission) are integrated implicitly because their coefficients times the sub-step exceed unity.

5 Code Classes and Data Structures

5.1 The two state vectors

Everything else follows from the choice of state.

ZrMicro: a flat array of length 19. Defined in `cpp_utils/parameters.h` and mirrored in `modelib_coupling.py`; the layout is Table 11.

Table 11: ZrMicro state vector. `N_PHYS=12`, `N_ACC=6`, `N_RHO=1`, `N_EQ=19`.

slice	contents	meaning	units
y[0:4]	C_v, C_i, C_{2i}, C_{3i}	mobile species	atom fraction
y[4:8]	$C_{iL}, C_{aiL}, C_{vL}, C_{avL}$	loop number densities	per atom
y[8:12]	$C_{iL}^i, C_{aiL}^i, C_{vL}^v, C_{avL}^v$	loop contents	per atom
y[12:18]	six accumulators	conservation diagnostics	per atom
y[18]	ρ_N	evolving network density	m^{-2}

The four loop families are the aligned/non-aligned split of the two loop types: iL and aiL are prismatic $\langle a \rangle$ interstitial loops, vL and avL basal $\langle c \rangle$ vacancy loops. The aligned fraction $f_a = (1 + 2f)/3$ carries the stress-induced preferential nucleation of Eq. (36), where f is the stress-dependent alignment fraction. At zero applied stress $f = 0$ and $f_a = 1/3$, so the three prismatic variants are equivalent.

Note. The notebook declares `SIGMA_N_PA = 0.0` and records it in the provenance manifest, but does *not* route it into `PARAMETER.OVERRIDES`. The 0-D reference run therefore inherits the workbook value $\sigma_n = 1 \times 10^8$ Pa, for which $f = 0.0518$ and $f_a = 0.3679$ rather than $1/3$. The manifest and the run disagree on this one quantity. It does not affect the `MoDeLib3` side, which reads its stress state from its own input files, nor the zero-stress verification of Route B; but a 0-D/3-D comparison that assumes equivalent $\langle a \rangle$ variants should add `sigma_n` to the override block.

MoDeLib3: two finite-element trial functions. Defined in `ClusterDynamicsFEM.h`:

```
typedef TrialFunction<'m',mSize,FiniteElementType> MobileTrialType; // mSize=4
typedef TrialFunction<'i',iSize,FiniteElementType> ImmobileTrialType; // iSize=8
```

The degrees of freedom are laid out as `index = node*iSize + component`, with components $[0, n_F)$ the densities n_k and $[n_F, 2n_F)$ the contents c_k , in family order $\{vL, a_1, a_2, a_3\}$. Note the family decomposition differs from the 0-D: `MoDeLib3` resolves the three crystallographically equivalent $\langle a \rangle$ prismatic variants explicitly, where the 0-D lumps them and splits by alignment instead. `modelib_export.py` performs the mapping.

5.2 Python classes

InputData (`input_data.py`). Owns three dictionaries — `material_params`, `physical_props`, `model_params` — mirroring the workbook sheets, plus `derived`, which holds everything computed from them. The derived quantities are the physics interface: jump frequencies $\omega_m = z_c \nu_m \exp(-E_m^m/k_B T)$, equilibrium concentrations, the length scales $\ell = z_c \Omega / (2\pi a^2)$, $\ell_a = \sqrt{\Omega/\pi b_a}$, $\ell_c = \sqrt{\Omega/\pi b_c}$, the generation-rate partition $G_v, G_i, G_{2i}, G_{3i}, G_{iL}, G_{aiL}, G_{vL}, G_{avL}$ satisfying $\sum G_x = G$, and the DAD capture efficiencies $Z_i^a, Z_v^a, Z_i^c, Z_v^c$ of Eqs. (63)–(64).

ReactionRates (`reaction_rates.py`). Holds no state of its own beyond a per-call cache. Its methods are the terms of the master equation, named after the reaction they represent: `R_i_v`, `R_i_i`, `R_i_2i`, `R_i_3i`, `R_2i_2i`, `R_2i_3i` for the binary reactions; `R_v_s`, `R_i_s` for the network sinks, built on the

sink *concentrations* of Eq. (66), $C_v^s = (a^2/z_c)\rho_N$ and $C_i^s = (a^2/z_c)Z_N\rho_N$; `loop_absorption` for Eqs. (83)–(89); `flux_v` and `flux_i` for the point-defect fluxes of Eq. (41),

$$\phi_v = \omega_v c_v, \quad \phi_i = \omega_i (c_i + 2c_{2i} + 3c_{3i}); \quad (2)$$

`nucleation_rate.*` for Eq. (36); `annealing_rate.*` for Eq. (38); and `a_shrink_gate` for the smooth minimum-size gate of Eq. (91).

`RateEquations` (`rate_equations.py`). Assembles those terms into dy/dt . One method per equation (`dCv_dt`, `dCiL_i_dt`, ...), plus `ode_system` which applies, in order: the positivity floor, the accumulator updates, the network density source, a non-negativity guard that zeroes any derivative driving a floored species further negative, and finally the `freeze_mobile` branch that zeroes `dydt[0:4]` — *last*, after every reaction term has already read the mobile values. That ordering is what makes the operator split exact rather than approximate.

`MoDELibFEMSolver` (`modelib_fem.py`). The adapter between the two codes. `write_sink_field()` converts the per-family (N, r) pairs into `MoDELib3` microstructure inputs, mapping the three $\langle a \rangle$ variants onto slip systems 6, 8, 10. `solve()` runs `DDomp` and `read_quadrature_mobile_field()` returns the mobile field at the quadrature points. `available()` reports whether either `DDomp` or `pyMoDELib` is built, which is what lets the notebook fall back to the analytic placeholder rather than fail.

5.3 C++ structures: **ZrMicro**

`Parameters` (`cpp_utils/parameters.h`). A flat aggregate of about 60 doubles, populated once from the command line so the right-hand side performs arithmetic only — no map lookups inside the integrator. It carries the jump frequencies $\omega_i, \omega_v, \omega_{2i}$; the Boltzmann factors $e_v = \exp(-E_v^F/k_B T)$, e_{2i} , e_{3i} and the stress-modified emission factors; the bias factors $Z_N, Z_i^a, Z_v^a, Z_i^c, Z_v^c$; the length scales ℓ, ℓ_a, ℓ_c ; and the eight generation rates. The `freeze_mobile` flag also lives here.

5.4 C++ classes: **MoDELib3**

`ClusterDynamicsParameters<dim>` (`ClusterDynamicsParameters.h`). Everything read from the material file, and every matrix derived from it. All members are `const` and initialized in the constructor’s member list, which matters: C++ initializes members in *declaration* order, and a member read by another member’s initializer must be declared first. (The `Eb` array was declared after `R1` but read by `getR1()`, an uninitialized read that silently dropped the Boltzmann factor from the cluster dissociation rates.) The principal members are collected in Table 12.

Its methods build the derived objects: `getR1()` and `getR2()` assemble the reaction matrices; `getLoopNucChannels()` selects the homogeneous clustering channels of Eq. (36); `clusterRadius()` and `clusterDensity()` implement the geometry of Eqs. (28)–(30), interpolating between the bi-pyramid law $\alpha_{bp}s_{bp}\pi r_{bp}n_{bp}$ of Eq. (14) and the planar loop law $2\pi r n$ of Eqs. (12)–(13) through the sigmoid of Eq. (30); and `loopDADbias()` evaluates Eq. (15),

$$Z_{aL,m}^{\text{DAD},0} = Z_m^0 \frac{p_m + p_m^{-2}}{2}, \quad Z_{cL,m}^{\text{DAD},0} = Z_m^0 p_m. \quad (3)$$

`ClusterDynamicsFEM<dim>` (`ClusterDynamicsFEM.h`). The numerical core. Its type aliases build the weak form compositionally from `MoDELib`’s expression templates: a trial function, its gradient, a flux matrix applied to that gradient, a bilinear form pairing test and trial, and a weak form

Table 12: Principal members of `ClusterDynamicsParameters`.

member	type	meaning
<code>msVector</code>	1×4 array	species sizes $\{-1, 1, 2, 3\}$
<code>D</code> , <code>invD</code> , <code>detD</code>	maps per grain	diffusion tensors, Eq. (10)
<code>G</code>	4×1	effective production rates
<code>Eb</code>	1×4	binding energies, Eq. (9)
<code>R1</code> , <code>R1cd</code>	4×4	first-order losses (sinks, dissociation)
<code>R2</code>	vector of 4×4	binary reactions
<code>loopNucChannels</code>	map	clustering channels, Eq. (36)
<code>immobileSpeciesVector</code>	1×4	family polarity
<code>immobileSpeciesBurgers</code>	3×4	Burgers directions, lattice components
<code>loopCascadeFractions</code>	1×4	ε_k , Eq. (51)
<code>nNuc</code>	1×4	defects per cascade-nucleated loop, Eq. (36)
<code>cLL</code> , <code>cLN</code>	1×4	coalescence coefficients, Eq. (99)
<code>kappaLL</code> , <code>kappaLN</code>	double	Avrami overlap, Eq. (98)
<code>tauVac</code>	double	vacancy-loop dissolution, Eq. (95)
<code>dadAnisotropy</code> , <code>dadZ0</code>	1×4	p_m , Z_m^0 of Eq. (15)
<code>loopSinkScale</code>	1×4	per-family sink calibration
<code>concentrationFloor</code>	double	positivity floor

integrating that bilinear form over the volume. Two linear solvers are held — a Cholesky solver `mSolver` for the pure diffusion operator, and a BiCGSTAB-backed `rSolver` for the Newton system, both wrapped in `FixedDirichletSolver` which eliminates the constrained degrees of freedom.

The public interface is four methods: `solveMobileClusters()` (the Newton iteration on Eq. (1)), `solveImmobileClusters()` (the pointwise dose march of Eq. (34) and Eq. (39)), `clampMobileClusters()` (the positivity floor, exempting the Dirichlet nodes), and `writeNodePositions()`.

EvalFunction specializations. Three small classes supply the terms that depend on the local field, each evaluated at quadrature points during assembly:

- **SecondOrderReaction** — the binary block. For a pair (a, b) it holds $-K_{ab}$ in both symmetric slots, so that $\mathbf{c}^T R_2[a] \mathbf{c} = -2K_{ab} c_a c_b$; the residual carries the compensating $\frac{1}{2}$ of Eq. (1), making the per-species loss rate $K_{ab} c_a c_b$.
- **ImmobileSinks** — the loop absorption sink acting on the mobile species, $k_m^2 = \sum_k Z(\text{type}(k), m) S_k$ from Eqs. (11)–(14), returned as $-k_m^2 \bar{D}_m$ on the diagonal. The grain-boundary sink is deliberately absent: unlike the 0-D reduction, which must lump it into a sink strength, the spatially-resolved model captures it through the Dirichlet condition of Eq. (19).
- **FluxMatrix** and **InvDscaling** — the diffusive flux $-D\nabla c/\Omega$ and the symmetrizing test-function scaling.

`ClusterDynamics<dim>` (`ClusterDynamics.h`). The outer wrapper: a `MicrostructureBase` subclass that plugs the cluster-dynamics field into `MoDELlib`’s microstructure container. It owns the parameters and the FEM object and implements the container interface — `solve()`, `getDt()`, `output()`, `updateConfiguration()` — together with the mechanical contributions (`averagePlasticDistortion`,

`stress, inelasticDisplacementRate`) through which the cluster field couples back to the elasticity solve.

5.5 Relationships among the structures

The dependency chain is short and strictly layered. In `MoDELib3`:

material file $\longrightarrow$ `ClusterDynamicsParameters` $\longrightarrow$ `ClusterDynamicsFEM` $\longrightarrow$ `{SecondOrderReaction, ImmobileSink}`

with `ClusterDynamics` wrapping the FEM object and exposing it to `DefectiveCrystal`. Nothing below the parameters object reads the material file, and nothing above the FEM object touches degrees of freedom directly.

In `ZrMicro` the corresponding chain is

workbook $\longrightarrow$ `InputData` $\longrightarrow$ `ReactionRates` $\longrightarrow$ `RateEquations` $\longrightarrow$ `cpp_bridge` $\longrightarrow$ `Parameters` $\longrightarrow$ `RHS`,

where the Python and C++ right-hand sides are deliberate mirrors: the Python one is the readable reference, the C++ one the production integrator, and they are required to agree.

The two chains meet only at `modelib_coupling` and `modelib_fem` in Route A, and only through the material definitions in Route B. That is the whole of the coupling surface, and keeping it that small is what has allowed the two codes to be reconciled term by term.

6 Changes and Differences from Upstream

Everything described so far is the architecture of `Zr3d_ghoniem`, the branch. This section separates what was inherited from upstream from what was added, so that the preceding sections are not read as a description of the upstream code.

6.1 Which upstream — a correction

Earlier revisions of this document compared `Zr3d_ghoniem` against `main` of `MoDELib2-NNL`. Those statements are individually accurate — a checkout confirms every row of Table 15 — but `MoDELib2-NNL` is the wrong baseline for cluster dynamics, and comparing against it makes the immobile solver look like an invention from nothing. It is not.

Table 13: The two upstream repositories. Both were checked out and read.

repository	iSize	immobile machinery
<code>mlm335/MoDELib2-NNL</code>	0	none: <code>empty solveImmobileClusters()</code> , no <code>ImmobileSinkRate.h</code> or <code>SpatialODESolver.h</code> , and a literal <code>// Missing immobile sinks placeholder</code>
<code>mlm335/MoDELib-fullCD</code>	8	complete: <code>ImmobileSinkRate.h</code> , <code>SpatialODESolver.h</code> , <code>FirstOrderReaction.h</code> , and <code>continuum-to-discrete loop conversion</code>

MoDELib-fullCD is the correct baseline. Measured against it, `Zr3d_ghoniem`’s immobile solver is a *re-discretization* of an existing scheme: 31 cluster-dynamics parameter keys are shared verbatim, and most of the remainder are renames (`loopNucDefects` $\rightarrow$ `nNuc`, `loopCoalLL/LN` $\rightarrow$ `cLL/cLN`,

Table 14: Immobile discretization, MoDELib-fullCD against Zr3d_ghoniem.

	MoDELib-fullCD	Zr3d_ghoniem
rate	Galerkin: assembled at quadrature points, then L^2 -projected through the consistent mass matrix (CG to 10^{-4})	nodal collocation; no mass matrix
time update	explicit Euler, then <i>sequential</i> implicit loss factors $(1 + dt\lambda_k)^{-1}$ per channel	one <i>fused</i> semi-implicit update
sub-cycling	none, one step of dtMax	nSub= 20 per dose step

loopAnnealTau0_SI→tau0_vLoop_SI, loopCoalNetwork_SI→rhoNetwork_SI, minimumLoopSize→r_min).

What differs is how the same physics is discretized:

Genuinely new in Zr3d_ghoniem: dadAnisotropy/dadZ0 (DAD reconciliation with the 0-D), atomicVolume_SI, concentrationFloor, loopSinkScale, size-dependent vacancy-loop emission, and the bi-pyramid family. Genuinely *lost*: clusterDiscretizationTime and initializeDiscreteClimbLoops(). MoDELib-fullCD converts its continuum loop field into discrete climb loops once the dose passes a threshold; Zr3d_ghoniem has no such transition, which is the largest functional regression of the fork and the capability its name refers to.

Note also that Zr4.txt in this checkout carries no immobile keys, but that is a property of one file and not evidence about upstream: MoDELib-fullCD's Zr4_Fitted.txt and Zr4_BMD19_nuc.txt each carry the full sixteen-key immobile set.

6.2 What MoDELib2-NNL contains

The remainder of this section is stated against MoDELib2-NNL, and is retained because it is the accurate account of *that* repository and of how much of the model had to be written on this branch without reference to MoDELib-fullCD (Table 15). Read it as the difference from the *dislocation-dynamics* upstream, not from the cluster-dynamics one.

Table 15: State of the immobile (loop) machinery in MoDELib2-NNL.

item	state on MoDELib2-NNL main
iSize	0 — no immobile species exist
solveImmobileClusters()	present, but an <i>empty function body</i>
ImmobileSinks.h	does not exist
mobile solve	carries the comment <code>// Missing immobile sinks</code>
Zr4.txt immobile block	absent; the file ends at <code>otherSinks_SI=0.0 0.0 0.0 0.0;</code>

MoDELib2-NNL therefore solves the mobile steady-state PDE of Eq. (1) — and nothing else. Diffusion, cascade production, the first-order losses R_1 and the binary reactions R_2 are all present and working. *The entire immobile half of the model, deliverable Sec. 2.2, was not implemented.*

Some scaffolding for it did exist. The geometry helpers clusterRadius(), clusterDensity(), sigmoid(), rpyr() and rloop() — Eqs. (28)–(30) — are all upstream code. With iSize=0, however, nothing ever called them.

6.3 Equations added

Table 16 lists the physics that had no upstream implementation, against the equation of the deliverable that each term realizes.

Table 16: Physics with no implementation in MoDELib2-NNL. All of it is new on the `zr3d_ghoniem` branch — though MoDELib-fullCD implements most of it too, by the different discretization of Table 14.

equation	content	implemented
Eq. (34)	loop density: nucleation – coalescence – dissolution	<code>solveImmobil</code>
Eq. (39)	loop content: cascade seed + absorption – coalescence – annealing – emission	<code>solveImmobil</code>
Eq. (36)	homogeneous SIA clustering nucleation Φ_{clus}	<code>getLoopNucCh</code>
Eqs. (11)–(14)	loop absorption sink on the mobile species	<code>ImmobileSink</code>
Eq. (15)	DAD capture efficiencies from p_m , Z_m^0	<code>loopDADbias(</code>
Eqs. (41)–(45)	absorption fluxes with the $ m_m $ cluster weighting	<code>solveImmobil</code>
Eq. (91)	minimum-stable-size gate on the shrinking channel	<code>solveImmobil</code>
Eq. (95)	vacancy-loop dissolution	<code>solveImmobil</code>
Eqs. (54)–(56)	size-dependent thermal vacancy emission	<code>solveImmobil</code>
Eqs. (97)–(102)	Avrami coalescence, two channels	<code>solveImmobil</code>

Two of these deserve a note on *how* they were added rather than merely that they were.

The loop absorption sink. `ImmobileSinks` is written as an `EvalFunction` mirroring the existing `SecondOrderReaction`, so it assembles into the mobile Newton system through the same mechanism as every other reaction term — `bWF.RI` into the Jacobian, `lWF.RI` into the residual. The sink set is the 0-D one *except* the grain boundary, which is imposed spatially by the Dirichlet condition of Eq. (19) and would otherwise be counted twice. The network dislocation sink required no new code: it reuses the existing `otherSinks.SI` plumbing, which upstream was simply set to zero.

The clustering nucleation source. Rather than being posited separately, Φ_{clus} is read off the reaction network the mobile solve already uses, so that the loops gain exactly what the mobile field loses. `getLoopNucChannels()` selects the pairs (a, b) whose reactants share polarity and whose product size $m_a + m_b$ is carried by no mobile species — the products that fall off the end of the truncated cluster ladder. For $\{v, i, 2i, 3i\}$ this selects $i + 3i$, $2i + 2i$ and $2i + 3i$. Those interstitials were previously unaccounted for: `getR2()` debits the reactants and credits nothing, which is the origin of the `Warning: Sum of R2 is not zero` message at start-up.

6.4 Corrections to pre-existing code

Four defects were found in code that was already there. They are listed separately from the additions because they affect the original code as distributed.

Binding energies read before initialization. `getR1()` reads the binding-energy array `Eb` to build the cluster dissociation rates, but `Eb` was declared *after* `R1` in the class. C++ initializes members in declaration order, not in the order of the constructor’s initializer list, so this was an uninitialized read: the Boltzmann factor $\exp(-E_b/k_B T)$ evaluated to unity instead of 3.8×10^{-9} , inflating the dissociation rate to the bare attempt frequency and suppressing C_{2i}

and C_{3i} by a factor 2.4×10^6 . **This is a defect of the original code, not of the present work.** It was invisible in the distributed calibration because every `Eb_eV` was set to 5 eV, for which $\exp(-5/k_B T) \sim 10^{-44}$ is negligible whether or not the factor is applied. Giving the di-interstitial its physical 0.957 eV is what exposed it. *It should be reported upstream:* it affects any MoDELib3 run with a non-negligible cluster binding energy.

The diffusion operator was scaled relative to the reactions. `FluxMatrix` divides D by `cdp.omega`, and the weak form multiplied the flux back by `ddBase.poly.Omega`. The two were the same quantity until the cluster-dynamics atomic volume became independently settable, after which the entire diffusion operator carried a spurious factor of `poly.Omega/cdp.omega` = 3.98 relative to every reaction term.

The minimum stable radius was never converted from metres. `r_min` is given in metres in the material file but compared against radii expressed in units of b , leaving the gate of Eq. (91) permanently open.

The atomic volume was the lattice *cell* volume. The polycrystal class returns `det(lattice basis)`. For a hexagonal lattice that cell contains two atoms, so the quantity is not the atomic volume the cluster-dynamics equations assume, and every content-radius-density conversion inherited the error. An optional material-file key, `atomicVolume_SI`, now supplies it explicitly, falling back to the previous behaviour when absent.

6.5 Input data: the mobile block

Po's `Zr4.txt` is left untouched — `main` still runs his standalone calibration unchanged. The reconciled definition is the separate file `Zr3d_ghoniem.txt`, whose mobile block resets seven keys and adds one (Table 17).

Table 17: Mobile-species parameters: original against reconciled.

key	Zr4.txt (original)	Zr3d_ghoniem.txt
<code>mobileSpeciesSurviving</code>	0.01	1.0
<code>Efficiency</code>		
<code>mobileSpeciesCascade</code>	1.0 0.6 0.3212 0.0788	0.995 0.97788714 0.005
<code>Fractions</code>		0.00333333
<code>mobileSpeciesEnergy</code>	1.5 3.0 6.0 8.0	1.6 3.0 6.0 8.0
<code>Formation_eV</code>		
<code>mobileSpeciesEnergy</code>	0.9 (v), 0.09 (i), 0.041–0.187	1.2 (v), 0.759101 (i, 2i, 3i);
<code>Migration_eV</code>	(2i, 3i); anisotropic	isotropic
<code>mobileSpeciesD0_SI</code>	4.9639e-10 (v), 8.1882e-10 (i, 2i, 3i)	5.21645e-7, all species
<code>reactionPrefactorMap</code>	2.856 for v+i, 1.0 or 0.0 elsewhere; 2i + 2i and 2i + 3i switched off	ten values derived from the 0-D jump frequencies; 2i+2i and 2i + 3i switched on
<code>otherSinks_SI</code>	0.0 0.0 0.0 0.0	1.88e14, 1.8988e14×3
<code>atomicVolume_SI</code>	<i>key did not exist</i>	1.2e-29

Three of these are worth quantifying. The original migration energies give D_i about $1200\times$ and D_{2i} about $2.5 \times 10^8\times$ the 0-D values at 573 K. The original `otherSinks_SI`=0 means there is *no*

network dislocation sink at all. And the original `mobileSpeciesCascadeFractions` do not partition the displacement rate the way the 0-D does: the reconciled values enforce $G_v + G_i + G_{2i} + G_{3i} + G_{iL} + G_{aiL} + G_{vL} + G_{avL} = G$.

The reaction prefactors are the one place where the two codes had to be made to agree algebraically rather than by substitution. `MoDELib3` forms $K_{ab} = p_{ab} 4\pi(r_a + r_b)(D_a + D_b)/\Omega$ while `ZrMicro` forms $K_{ab} = f_{ab}(\omega_a + \omega_b) = f_{ab}(z_c/a^2)(D_a + D_b)$, so

$$p_{ab} = f_{ab} \frac{z_c}{a^2} \frac{\Omega}{4\pi(r_a + r_b)}, \quad (4)$$

with $f_{ab} = 0.5$ for v+i (the 0-D recombination factor) and unity otherwise. Pairs involving 2i or 3i additionally carry the ratio of diffusivities that restores the ω_{2i} -based 0-D rate, because those species transport at the monomer diffusivity here.

6.6 Input data: the immobile block

There is no “before” column for this one: the twenty-odd keys of Table 18 are all new, because none of the immobile machinery existed.

Table 18: Immobile-species keys, all added. None has a counterpart in `Zr4.txt`.

group	keys
family definition	<code>immobileSpeciesVector</code> , <code>immobileSpeciesRelRelaxVol</code> , <code>immobileSpeciesBurgers</code>
nucleation	<code>loopCascadeFractions</code> (ε_k), <code>nNuc</code> (n_k^{nuc}), <code>evc</code> , <code>Nvmax</code>
geometry	<code>alpha_bp</code> , <code>delVPyramid</code> , <code>w0</code> , <code>n_s</code> , <code>nmin</code> , <code>nmax</code> , <code>r_min</code>
coalescence	<code>cLL</code> , <code>cLN</code> , <code>kappaLL</code> , <code>kappaLN</code>
dissolution and emission	<code>tau0_vLoop_SI</code> , <code>Ea_vLoop_eV</code> , <code>Eb_eV</code>
sinks and bias	<code>rhoNetwork_SI</code> , <code>dadAnisotropy</code> , <code>dadZ0</code> , <code>loopSinkScale</code>
numerics	<code>concentrationFloor</code>

Two of these are calibration choices rather than physical constants, and are flagged here because they are easy to mistake for the latter.

`loopSinkScale` reproduces `ZrMicro`’s sink convention exactly. The 0-D applies the $\langle c \rangle$ radius prefactor $\ell_c = \sqrt{\Omega/\pi b_c}$ to *all four* families — ℓ_a is computed and never used — whereas the 3-D builds r_k from each family’s own $|b_k|$. Each family therefore needs $\sqrt{|b_k|_{3D}/b_c^{0D}}$, and the $\langle c \rangle$ channel additionally carries the fitted $Q = 0.287931$. Setting all four to unity recovers the purely geometric form.

`dadAnisotropy` and `dadZ0` are fitted, not derived. They are obtained by closed-form inversion of Eq. (15) against the 0-D symmetric forms Eqs. (63)–(64), exactly, to a residual of 10^{-16} . The cost is that the fitted p_m is *not* the anisotropy the migration tensors imply, so these runs do not exercise the tensorial-DAD mechanism that is the distinctive physics of the 3-D model; Eq. (15) is acting as a two-parameter fitting form. The fit is also tied to $T = 573$ K, because δ_i is temperature-interpolated in the 0-D.

6.7 What is unchanged

Of the eighteen files that differ from `main`, most are additions. Nothing outside the cluster-dynamics path was touched: elasticity, mobility laws, gamma-surface, the discrete-dislocation machinery, the

mesh and FEM infrastructure, and the microstructure generator are all as distributed. Within the cluster-dynamics path, the mobile transport operator, the R_1 and R_2 assembly, the trial-function and weak-form types, and the linear solvers are Po's — the changes to them are the four corrections above, not redesigns.

Two files in the branch are *not* intended for upstream: `CMakeLists.txt` and `tools/CMakeLists.txt` carry local-environment adaptations (Eigen include path moved from MacPorts to a Linux location, the maintainer's Qt path removed, and the `DDQt` target disabled because Qt is absent here). They are correct for a Linux or WSL build and would break a macOS build, and should be dropped from any pull request.

A Code Manual

This appendix is operational: what to install, what to run, what to change, and what to check when something goes wrong.

A.1 Prerequisites

Table 19: Environment.

component	version	note
Python	3.14	virtual environment <code>Fluor.Zr/.Zr_venv/</code>
Jupyter kernel	<code>fluor_zr</code>	registered as “Python 3.14 (Fluor.Zr)”
SUNDIALS	7.1.1	<code>Libraries/sundials-7.1.1/</code> ; CVODE and ARKODE
CMake	$\geq 3.16, < 4$	CMake 4 rejects <code>cmake_minimum_required(3.1)</code>
Eigen	3.4	MoDELib3 dependency
FFTW	3	<code>libfftw3-dev</code> ; MoDELib3 will not configure without it
compiler	C++17 / C++20	ZrMicro needs C++17, MoDELib3 needs C++20

Do not use Anaconda Python for this project: its NumPy conflicts with the SciPy build in the virtual environment.

On Windows, MoDELib3 is built and run inside WSL. Its `DDomp` is a Linux ELF and is invoked through `ws1.exe`, with Windows paths translated to `/mnt/<drive>/...` ZrMicro builds natively on Windows.

A.2 Building

ZrMicro C++ solver.

```
cd ZrMicro/cpp_utils
cmake -S . -B ../build -DCMAKE_BUILD_TYPE=Release
cmake --build ../build --config Release
```

Use `Release`: the parameter fit calls the solver thousands of times, and the OpenMP batch mode is only worth having when optimized. CMake auto-detects OpenMP; the build still works serially without it.

MoDELib3.

```
cd <DisloCluster>/MoDELib3/build
cmake -S . -B . -DCMAKE_BUILD_TYPE=Release
cmake --build . --target DDomp -j 12
```

The build takes 10–30 minutes from clean. Two files on the `zr3d_ghoniem` branch are local-environment adaptations and would break a macOS build: `CMakeLists.txt` (Eigen path, Qt path removed) and `tools/CMakeLists.txt` (DDqt disabled). Drop both from any pull request to upstream.

A.3 Running

Route A — coupled march. Open `ZrMicro/code/coupled_0d_3d_ZrMicro.ipynb` on the `dislocluster` kernel and run all cells; or headless:

```
.DisloClusterVenv\Scripts\python.exe -m nbconvert --to notebook --execute \
--ExecutePreprocessor.kernel_name=dislocluster \
ZrMicro/code/coupled_0d_3d_ZrMicro.ipynb
```

Everything the user is expected to change is in cell 12. Before the first coupled run the simulation directory must be seeded once: `seed_sim_dir_from_tutorial(MODELIB_ROOT, SIM_DIR)`, then run its `generateInputFiles.py`. Until that is done the notebook silently falls back to the analytic placeholder — it says so loudly in its output, and that message should be read every time.

Route B — monolithic MoDELib3 run.

```
cd <DisloCluster>/MoDELib3/tutorials/zrmicro_coupled
./clean_run.sh
```

The script refreshes the material file and the mesh from `Library/` (LF-converted), regenerates `evl_0`, then runs `microstructureGenerator` and `DDomp`. A 31-step run to 30 dpa takes about one hour on 24 cores. Output appears in `evl/` as `evl_N.txt` plus `cdNodes.txt`.

0-D only. `ZrMicro/code/ZrMicro.ipynb` runs the 0-D model alone and produces the full figure suite plus `provenance.md` in a timestamped output directory.

A.4 Post-processing

```
from py_utils.modelib_fields import plot_field_panels
from py_utils import modelib_gb as gb

plot_field_panels(EVL, steps=[0,4,9,29], doses=[1,5,10,30],
                  species=("Cv","Ci","C2i","C3i"), out_file=...)
gb.plot_density_profiles(EVL, steps, doses, max_nm=150.0, n_bins=45,
                        out_file=...)
```

Output is written *after* `solve()`, so `evl_N` holds the state at $N + 1$ dose steps — `evl_0` is 1 dpa, not 0. Loop diameters must be formed from binned mean content and density, never by averaging nodal diameters: size is a ratio of two fields, and the mean of a ratio is not the ratio of the means.

A.5 Changing things

Note that the temperature appears in *three* places — the workbook, the notebook override, and `polycrystal.txt` — and that `dadAnisotropy/dadZ0` are fitted at 573 K. Changing T requires refitting them, or the DAD bias will be silently wrong.

Table 20: Where to change what.

to change	edit
irradiation conditions (T, G, σ)	cell 12 of the notebook <i>and</i> <code>polycrystal.txt</code> / <code>Zr3d_ghoniem.txt</code> ; they are not linked
0-D material parameters	the workbook, or <code>PARAMETER_OVERRIDES</code> (which wins)
3-D material parameters	<code>Library/Materials/Zr3d_ghoniem.txt</code> , then re-run <code>clean_run.sh</code> so the copy in <code>inputFiles/</code> is refreshed
dose schedule (Route A)	<code>DOSE_START</code> , <code>DOSE_INTERVAL</code> , <code>DOSE_MAX</code>
dose schedule (Route B)	<code>Nsteps</code> and <code>dtMax</code> in <code>DD.txt</code>
mesh or domain size	<code>meshFile</code> and <code>F</code> in <code>polycrystal.txt</code>
number of loop families	<code>iSize</code> in <code>ClusterDynamicsParameters.h</code> , plus every <code>1, iSize/2</code> array in the material file
0-D solver method	<code>solver_method</code> in <code>SOLVER_CONFIG</code> ; <code>CVODE BDF + dense</code> is the recommended default

A.6 Troubleshooting

The governing rule of this stack: almost every failure exits zero. The MoDElib3 build script, `DDomp` with a missing `evl_0`, a missing material key, a CRLF mesh, and several solver failures all return status 0. Verify by artifacts on disk and by log tails, never by exit status.

Table 21: Symptoms and causes.

symptom	cause and fix
<code>microstructureGenerator</code> never terminates	It is the discrete-dislocation path, inserting loops one at a time; CD loop densities need $\sim 10^{11}$ of them. Set <code>useDislocations=0</code> and <code>targetDensity=0</code> .
Loop content comes out at exactly $3\times$ the expected value	<code>startAtTimeStep=-1</code> resumed from a stale <code>evl_N</code> and double-counted dose. Pin <code>startAtTimeStep=0</code> .
Parser reports zero mesh nodes, no error	CRLF line endings in the <code>.msh</code> . <code>clean_run.sh</code> LF-converts it on every run for this reason.
Newton residual stalls near 10^5	An initial-guess problem, not a linear-algebra one. The direct <code>SparseLU</code> branch was tried and fails in <code>compute()</code> on the $\sim 96\text{k}$ -dof matrix; the iterative branch is the working one.
Run restarts from the previous dose	<code>evl_0.txt</code> is both the initial configuration and the <code>runID 0</code> output. Regenerate it every run.
Notebook silently produces analytic profiles	The fast-solve backend fell back to <code>placeholder</code> . Check whether <code>DDomp</code> is built and <code>SIM_DIR/inputFiles/</code> is seeded.
C_{2i} , C_{3i} orders of magnitude low	Historically the Eb-before-R1 declaration-order bug. Fixed on this branch; if it reappears, check that no member's initializer reads a member declared after it.
Three $\langle a \rangle$ families give different densities at zero stress	Burgers vectors supplied in Cartesian rather than lattice components. The zero-stress symmetry check is the fastest detector of this class of error.

A.7 Reproducibility

Every run writes a timestamped, git-tagged directory `output/<YYYYMMDD_HHMMSS>_<git-hash>/` containing the figures and `provenance.md`, the latter recording every input parameter, the software environment, and the solver statistics. The 0-D reference used throughout this work is `output/20260622_144021_7959445`.

One caution on reproducing it: `provenance.md` stores values to four significant figures only, so re-reading it will *not* reproduce the run. The full-precision fitted set lives in the `PARAMETER_OVERRIDES` block of the notebook, which is why that block is pinned rather than inherited from the workbook.